



BUNNIES & TEA

IRENE DIGES GARCÍA
2ºDAW



TÍTULO DEL PROYECTO: BUNNIES & TEA.

AUTOR: IRENE DIGES GARCÍA.

CURSO: 2º DAW, AÑO 25/26.

MÓDULO: PROYECTO INTERMODULAR II

GITHUB: [HTTPS://GITHUB.COM/CREEPYKAI/BAT](https://github.com/creepykai/bat)

Contenido

1	Introducción.....	3
1.1	Idea general	3
2	Diseño del proyecto	4
2.1	Objetivos, alcance y viabilidad.....	4
2.1.1	Objetivo principal	4
2.1.2	Objetivos específicos	4
2.1.3	Alcance del proyecto	5
2.1.4	Viabilidad.....	5
2.2	DAFO y elección de herramientas	6
2.3	Herramientas	6
3	Documentación del diseño técnico	7
3.1	Diseño de base de datos.....	7
3.1.1	Diccionario de datos.....	8
3.2	Diseño de clases	9
3.3	Casos de uso	9
3.4	MockUp	12
3.4.1	Título.....	12
3.4.2	Configuración	12
3.4.3	Juego principal	13
3.4.4	Tienda.....	13
4	Planificación de actuaciones	14
4.1	Secuenciación de actividades e identificación de tareas	14
4.2	Estimaciones temporales	14
4.3	Procedimientos de ejecución.....	15
4.4	Recursos y valoración económica.....	15
4.4.1	Recursos materiales.....	15
4.4.2	Recursos humanos.....	15
4.4.3	Coste estimado	16

4.4.4	Financiación	16
4.4.5	Consideraciones para un lanzamiento profesional	16
4.5	Aspectos legales y licencias	16
4.6	Gestión de riesgos.....	17
5	Seguimiento y evaluación	18
5.1	Evaluación	18
5.1.1	Evaluación inicial	18
5.1.2	Planificación de las revisiones	18
5.2	Indicadores de calidad	19
5.2.1	Batería de procedimientos de pruebas	19
5.3	Gestión de incidencias	20
5.3.1	Sistematización y temporalización	20
5.4	Gestión de cambios	20
5.4.1	Formato de registro de cambios	20
5.4.2	Análisis de impacto	20
5.5	Desarrollo Técnico e Implementación	21
5.5.1	Backend y Lógica de Negocio.....	21
5.5.2	Frontend e Interfaz de Usuario	21
5.5.3	Ampliación de contenido y sistema de logros	22
5.6	Manual de uso y Despliegue del sistema	22
5.6.1	Manual para usuario	22
5.6.2	Manual de administrador	22
6	Evaluación y conclusiones finales	23

1 INTRODUCCIÓN

El proyecto Bunnies & Tea consiste en el diseño y desarrollo de una aplicación web interactiva bajo la modalidad de idle game o juego de progresión pasiva. El sector de los "cozy games" (juegos tranquilos y estéticos) ha crecido exponencialmente, y este proyecto busca aplicar tecnologías de desarrollo en entorno servidor y cliente para ofrecer una experiencia relajante y visualmente atractiva.

Técnicamente, el proyecto se apoya en una arquitectura donde la seguridad y la persistencia continua de los datos son clave. Para lograrlo, la aplicación integra un sistema de sesiones en el servidor, comunicación asíncrona entre cliente y servidor (AJAX) y el uso de bases de datos relacionales para el guardado automático del progreso. De forma complementaria, se mantiene la gestión mediante ficheros de estado (.state) como opción de respaldo local para el usuario. Todo este ecosistema se encuentra orquestado e implementado dentro de un entorno de contenedores Docker.

1.1 IDEA GENERAL

Mi proyecto se llama Bunnies & Tea y está pensado como una aplicación web tipo idle game. La idea es que sea un juego relajante, donde tienes una pequeña cafetería de conejos que produce tazas de té y pasteles automáticamente.

La jugabilidad será sencilla: habrá una producción automática que vende té o bollos (aleatoriamente de los que tengas desbloqueados) cada segundo. Habrá también un botón de clic en la pantalla de juego para que el jugador gane monedas extra si hace clic.

Con las monedas, el jugador irá a la Tienda y podrá desbloquear cosas. Principalmente recetas (nuevos té/pasteles) y decoraciones, que funcionarán como multiplicadores que afectan solo a algunas recetas (por ejemplo, una decoración de "tetera" mejora el precio de los té).

La aplicación tendrá cuatro páginas principales: una Entrada (que ahora incluirá un sistema de registro e inicio de sesión para identificar a cada usuario), la pantalla de Juego (la cafetería), la Tienda y los Ajustes. Gracias al sistema de sesiones, el progreso del jugador se guardará automáticamente en el servidor. En Ajustes, el jugador podrá descargar un fichero .state con su progreso a modo de copia de seguridad local, o usarlo si quiere restaurar una partida antigua.

El proyecto usará las herramientas de clase: PHP, HTML, CSS, JavaScript y MariaDB, todo dentro de Docker con Apache. PHP se encargará de gestionar la base de datos, las sesiones y validar toda la lógica en el servidor para evitar trampas (guardar las compras, ingresos reales). Por su parte, JavaScript controlará los contadores en tiempo real y el botón de clic, comunicándose con PHP de forma asíncrona (AJAX) para que el juego se actualice sin tener que recargar la página.

Para el diseño, usaré dibujos superpuestos (GIFs o PNGs) que haré yo sobre un fondo estático (JPG).

2 DISEÑO DEL PROYECTO

2.1 OBJETIVOS, ALCANCE Y VIABILIDAD

2.1.1 Objetivo principal

El jugador puede mejorar su cafetería, desbloquear nuevas recetas y contratar más conejos para aumentar la producción y el valor de cada producto.

El proyecto busca transmitir una sensación de progreso tranquilo y sin presión, mientras se demuestra la capacidad de desarrollar una aplicación web completa y segura, integrando el uso de sesiones en PHP, bases de datos relacionales y comunicación asíncrona (AJAX) para lograr una experiencia interactiva fluida entre el cliente y el servidor.

2.1.2 Objetivos específicos

En cuanto a los objetivos técnicos, mi prioridad es programar una lógica robusta en PHP estructurada bajo una arquitectura MVC (Modelo-Vista-Controlador) que no solo calcule la producción automática basándose en los conejos y recetas desbloqueadas, sino que valide cada acción en el servidor para evitar cualquier tipo de manipulación o trampa desde el navegador. Para asegurar el progreso de manera profesional, implementaré un sistema de autenticación con sesiones que conecte a cada usuario con su partida, guardando todo (monedas, inventario y mejoras) de forma continua en una base de datos MariaDB mediante el uso de transacciones SQL seguras para garantizar la integridad de los datos en cada compra o cambio de estado.

El sistema de ficheros JSON (.state) lo mantendré, pero evolucionará para ser una herramienta secundaria de copia de seguridad local. Además, garantizaré que el intercambio de datos entre el cliente (JavaScript) y el servidor (PHP) sea fluido y sin recargas utilizando peticiones asíncronas (AJAX), algo fundamental para que el botón de clicker, los contadores y los avisos dinámicos de penalización funcionen en tiempo real.

Por otro lado, mis objetivos de interfaz y experiencia se centran en hacer que la cafetería sea un entorno dinámico y agradable. Añadiré pequeñas animaciones mediante CSS y JS, como el movimiento de los conejos o el vapor de las tazas de té, para que la pantalla de juego se sienta "viva". Asimismo, se integrará un sistema de feedback visual que alerte al jugador sobre el estado de limpieza y rendimiento de la tienda, incentivando la interacción constante. Me aseguraré de que la aplicación sea totalmente funcional y responsiva en diferentes navegadores y dispositivos, manteniendo siempre una coherencia visual impecable.

Finalmente, a nivel de gestión, cumpliré con todos los requisitos técnicos del ciclo formativo desplegando el proyecto en el entorno de contenedores Docker con Apache, tal como trabajamos en clase. Para asegurar la calidad del proyecto, organizaré el código de manera modular siguiendo el patrón MVC, lo que me facilitará enormemente el trabajo cuando quiera añadir más contenido en el futuro, como nuevas recetas o tipos especiales de conejos.

2.1.3 Alcance del proyecto

El alcance de este proyecto abarca el desarrollo de varias pantallas principales y la infraestructura técnica necesaria para que todo el sistema funcione de forma segura y conectada. En primer lugar, desarrollaré una pantalla de acceso y registro (login.php y registro.php) para que cada jugador pueda crear su cuenta de forma segura y cargar su partida directamente desde la base de datos. Una vez dentro, la interfaz principal será la pantalla de juego (juego.php), donde ocurrirá la producción automática, se mostrarán los contadores en tiempo real y estará el botón de "clicker". Para que la experiencia sea continua, los avances se guardarán en el servidor automáticamente mediante peticiones en segundo plano.

También incluiré una pantalla de tienda (tienda.php) donde el jugador podrá gastar sus monedas para desbloquear nuevas recetas y decoraciones (multiplicadores). En esta sección, el sistema siempre comprobará el saldo real desde el backend antes de confirmar cualquier compra. Además, desarrollaré una pantalla de ajustes (ajustes.php) para que el usuario pueda cerrar sesión o, si lo desea, descargar y subir su progreso en un fichero .state a modo de copia de seguridad local. Todo este sistema estará respaldado por una base de datos relacional en MariaDB, con tablas diseñadas para almacenar a los usuarios con contraseñas cifradas, el catálogo completo de objetos y el inventario exacto de cada jugador.

Por el contrario, el proyecto tiene unos límites definidos para ajustarse a los plazos del ciclo: no incluirá funciones multijugador, la posibilidad de visitar la cafetería de otros jugadores ni clasificaciones globales. Tampoco integraré animaciones complejas, uso de inteligencia artificial o pasarelas de pago reales, ya que toda la economía de la tienda se gestionará de forma simulada y exclusivamente con las monedas ficticias del propio juego.

2.1.4 Viabilidad

El proyecto es totalmente viable para realizarlo de forma individual, ya que la lógica de juego se basa en cálculos simples y, aunque ahora la base de datos gestionará el registro de usuarios y el guardado automático de la partida, sigue siendo pequeña y manejable.

Se puede desarrollar por fases: primero me centraré en la parte funcional (el login, la conexión con la base de datos, la producción, monedas y mejoras validadas en PHP para evitar trampas) y después pasaré a la parte visual (interfaz, animaciones y estética).

Todo se puede hacer con los conocimientos vistos en clase, usando peticiones asíncronas para que el juego se actualice sin recargar la página, y sin usar tecnologías externas. Además, el juego tiene un estilo propio y diferente al resto, lo que hace que sea un proyecto llamativo, original y perfectamente defendible en la presentación final.

2.2 DAFO Y ELECCIÓN DE HERRAMIENTAS

Fortalezas	Debilidades
Es una idea diferente, original y visualmente atractiva, que se sale de los proyectos típicos. Combina bien la parte técnica (PHP con sesiones de usuario, base de datos MariaDB y peticiones asíncronas en JavaScript) con una propuesta creativa. Además, tiene una estructura modular que se puede ampliar fácilmente (más recetas, conejos o decoraciones).	Puede requerir algo de tiempo para ajustar bien los valores de producción y las recompensas, ya que toda la lógica debe validarse desde el servidor para evitar trampas. Es importante controlar los recursos visuales (animaciones o imágenes) para no sobrecargarlo. Al ser un juego, no tiene un objetivo “útil” más allá del entretenimiento, aunque sí demuestra habilidades técnicas avanzadas.
Oportunidades	Amenazas
Añadir nuevos tipos de conejos con distintas habilidades (baristas, pasteleros, recolectores...). Incluir recetas especiales que aumenten el valor del té y hagan aparecer tazas premium aleatorias. Incorporar logros, estadísticas o decoraciones estacionales en el futuro. Por último, la separación que he diseñado entre cliente y servidor abre la posibilidad de convertirlo más adelante en una app móvil.	Falta de tiempo para pulir los detalles visuales o añadir todas las mejoras planificadas. Dificultad para equilibrar las mecánicas si se añaden demasiados elementos nuevos. Posibles errores al guardar o cargar datos si no se controlan bien las actualizaciones del progreso y la concurrencia de peticiones a la base de datos.

2.3 HERRAMIENTAS

Para el desarrollo de Bunnies & Tea, utilizaré **PHP** como lenguaje principal en el lado del servidor. Su función será gestionar toda la parte lógica del juego, desde el control de las monedas y el inventario hasta la validación de las compras en la tienda para evitar manipulaciones. Además, PHP será el encargado de gestionar las **sesiones de usuario**, permitiendo que cada jugador acceda a su progreso de forma segura. Como motor de persistencia, usaré **MariaDB** para almacenar toda la información del progreso en tablas relacionales que conecten a los usuarios con sus recetas, conejos y decoraciones.

En cuanto al frontend, emplearé **HTML y CSS** para montar la interfaz y el diseño visual responsivo. **JavaScript** será fundamental para que el juego se sienta fluido; lo usaré para mostrar los contadores y las pequeñas animaciones en tiempo real sin tener que recargar la página. Para lograr una integración perfecta entre el cliente y el servidor, implementaré **peticiones asíncronas (AJAX)**, permitiendo que JavaScript comunique los cambios al servidor de forma transparente para el usuario.

El proyecto se ejecutará sobre un servidor **Apache dentro de Docker**, replicando exactamente el entorno de contenedores que utilizamos en clase para asegurar que el despliegue sea idéntico en cualquier equipo. Para la gestión del código y el control de versiones, utilizaré **Git y GitHub**, lo que me permitirá llevar un historial detallado de los avances y contar con copias de seguridad remotas.

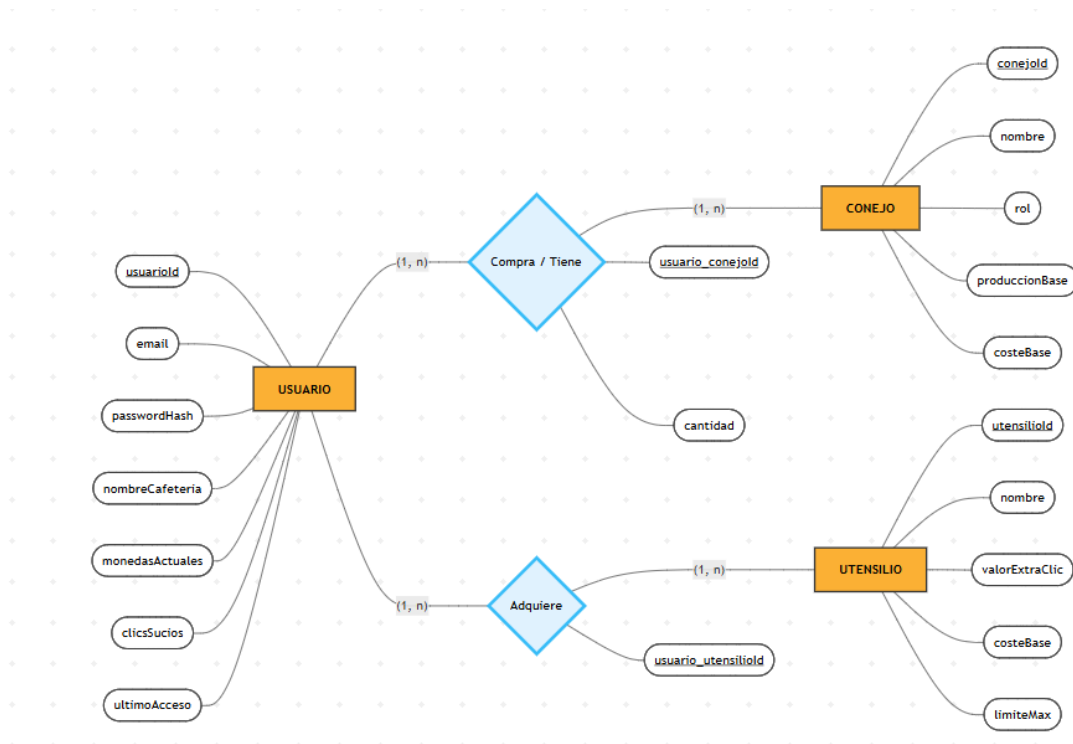
El código fuente y el historial de versiones se encuentra alojado en el siguiente [repositorio](#).

En la parte creativa, usaré **MediBang Paint** para dibujar todos los recursos visuales originales (GIFs y PNGs), manteniendo un estilo propio. Finalmente, si el desarrollo permite incluir estadísticas avanzadas de producción en el futuro, integraré la librería **Chart.js**, ya que ofrece una visualización de datos limpia y sencilla de implementar.

3 DOCUMENTACIÓN DEL DISEÑO TÉCNICO

3.1 DISEÑO DE BASE DE DATOS

El sistema de persistencia de **Bunnies & Tea** utiliza una base de datos relacional MariaDB gestionada mediante contenedores Docker. El diseño se ha estructurado para soportar un sistema multiusuario seguro, utilizando **camelCase** para la nomenclatura de todos los elementos y tipos de datos precisos para evitar errores en los cálculos de la economía del juego.



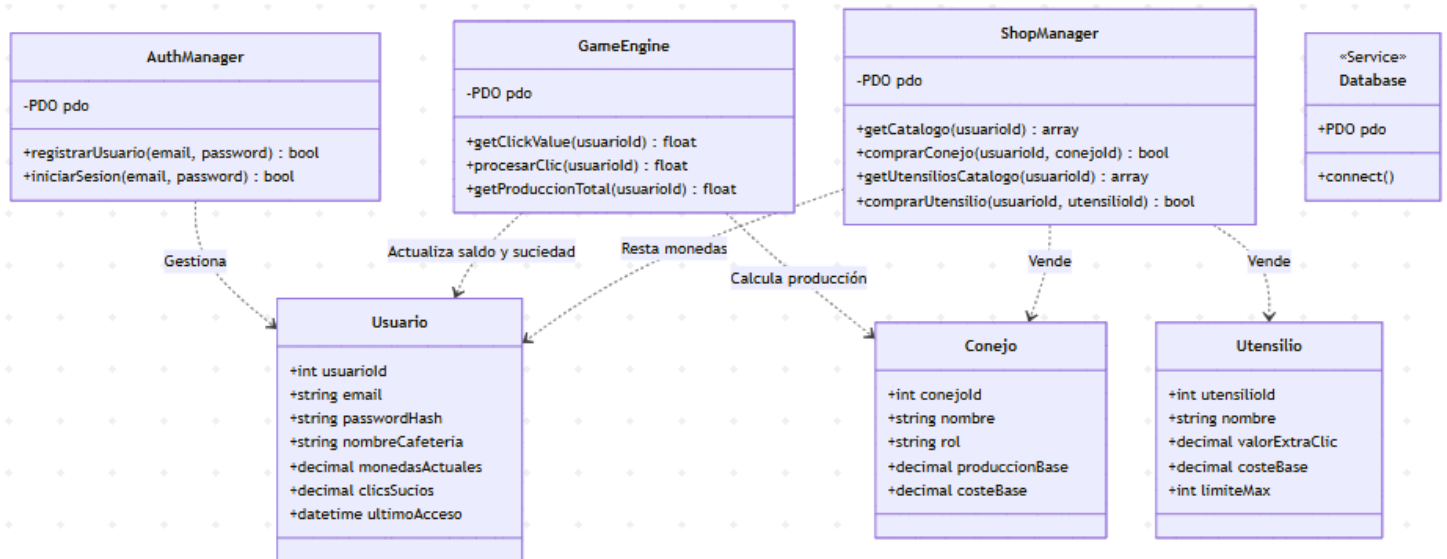
3.1.1 Diccionario de datos

- **Tabla: usuario** Centraliza las cuentas y el estado financiero global de cada jugador.
 - `usuariold` (INT, PK, AI): Identificador único del jugador.
 - `email` (VARCHAR, Unique): Correo electrónico para el inicio de sesión.
 - `passwordHash` (VARCHAR): Contraseña cifrada para garantizar la seguridad del acceso.
 - `nombreCafeteria` (VARCHAR): Nombre personalizado de la instancia del juego.
 - `monedasActuales` (DECIMAL 20,2): Saldo disponible para compras.
 - `clicsSucios` (DECIMAL 10,2): Registro de la suciedad acumulada por la actividad, utilizada para calcular las penalizaciones de la cafetería.
 - `ultimoAcceso` (DATETIME): Registro de la última conexión para calcular la producción pasiva fuera de línea.
- **Tabla: conejo** Catálogo maestro de los empleados disponibles en la tienda.
 - `conejold` (INT, PK, AI): Identificador del tipo de conejo.
 - `nombre` (VARCHAR): Nombre del tipo (ej. Conejo Barista).
 - `rol` (VARCHAR): Función técnica dentro de la cafetería.
 - `produccionBase` (DECIMAL 10,2): Monedas por segundo que genera por unidad.
 - `costeBase` (DECIMAL 15,2): Precio inicial en la tienda.
- **Tabla: usuario_conejo** Tabla asociativa de inventario de personal.
 - `usuario_conejold` (INT, PK, AI): Identificador del registro.
 - `usuariold` (INT, FK): Referencia al dueño.
 - `conejold` (INT, FK): Referencia al tipo de conejo.
 - `cantidad` (INT): Número de unidades de este tipo que posee el jugador.
- **Tabla: utensilio** Catálogo maestro de las mejoras aplicables a los clics manuales del jugador.
 - `utensiliold` (INT, PK, AI): Identificador de la mejora.
 - `nombre` (VARCHAR): Nombre del objeto (ej. Tetera Dorada).
 - `valorExtraClic` (DECIMAL 10,2): Monedas adicionales que suma al saldo por cada clic manual.
 - `costeBase` (DECIMAL 15,2): Precio inicial en la tienda.
 - `limiteMax` (INT): Cantidad máxima que un jugador puede poseer de este objeto.
- **Tabla: usuario_utensilio** Tabla asociativa de inventario de mejoras desbloqueadas.
 - `usuario_utensiliold` (INT, PK, AI): Identificador del registro.
 - `usuariold` (INT, FK): Referencia al dueño.
 - `utensiliold` (INT, FK): Referencia a la mejora adquirida.

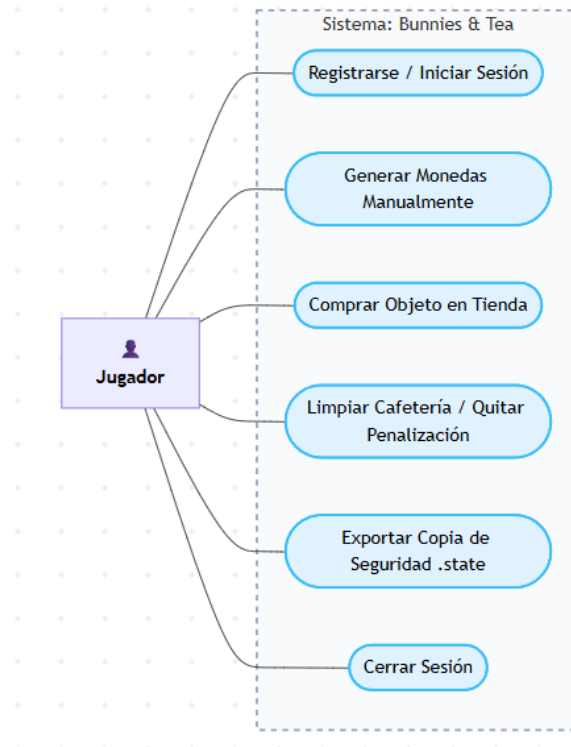
3.2 DISEÑO DE CLASES

Para la arquitectura del backend en PHP, he optado por un diseño basado en Programación Orientada a Objetos (POO) y estructurado bajo el patrón de diseño arquitectónico MVC (Modelo-Vista-Controlador). Esto garantiza la modularidad del código, facilita el mantenimiento y asegura la integridad de los datos mediante la encapsulación (usando modificadores de visibilidad private y métodos public).

El sistema se divide en las siguientes clases principales:



3.3 CASOS DE USO



Caso de Uso: CU-01: Registrarse / Iniciar Sesión

Actor(es):	Jugador
Descripción:	El jugador crea una cuenta segura o accede a su perfil para recuperar su progreso desde la base de datos MariaDB.
Precondiciones:	El usuario está en la pantalla de entrada (index.php).
Flujo Principal:	1. El Jugador introduce su email y contraseña. 2. El Sistema (PHP) cifra la contraseña (password_hash) si es registro, o la verifica si es login. 3. El Sistema genera una sesión de usuario segura (\$_SESSION). 4. El Sistema recupera el estado completo de la cafetería desde la BBDD y redirige a juego.php.
Postcondiciones:	El Jugador está autenticado y visualiza su partida exacta.

Caso de Uso: CU-02: Generar Monedas Manualmente (Click)

Actor(es):	Jugador
Descripción:	El jugador gana monedas interactando con el botón principal. El saldo se sincroniza en segundo plano.
Precondiciones:	Jugador autenticado en la pantalla principal (juego.php).
Flujo Principal:	1. El Jugador hace clic en el botón de "clicker". 2. El Sistema (JavaScript) actualiza el contador visual de inmediato para que la experiencia sea fluida. 3. El Sistema envía una petición asíncrona (AJAX) al servidor (PHP) con el clic. 4. El servidor (PHP) actualiza el campo monedasActuales en MariaDB para ese usuariold.
Postcondiciones:	El saldo de monedas aumenta tanto en la pantalla como en la BBDD de forma segura.

Caso de Uso: CU-03: Comprar Objeto en Tienda

Actor(es):	Jugador
Descripción:	El jugador gasta su saldo para adquirir nuevas recetas, conejos o multiplicadores. La transacción se valida estrictamente en el backend.
Precondiciones:	Jugador autenticado en la pantalla de la tienda (tienda.php).
Flujo Principal:	1. El Jugador pulsa "Comprar" sobre un objeto (ej. un conejo). 2. El Sistema (JavaScript) envía una petición AJAX al servidor con el ID del objeto. 3. El servidor (PHP) consulta en la BBDD el precio real del objeto y el saldo real del jugador. 4. Flujo A (Éxito): Si tiene saldo, PHP descuenta las monedas, añade el objeto a usuarioConejo, y devuelve un mensaje de éxito. La interfaz se actualiza. 5. Flujo B (Fallo): Si no tiene saldo, PHP bloquea la transacción y devuelve un error. La interfaz avisa al jugador.
Postcondiciones:	(Si Flujo A) El jugador adquiere el objeto y su saldo se reduce en la base de datos. (Si Flujo B) El estado del juego no cambia.

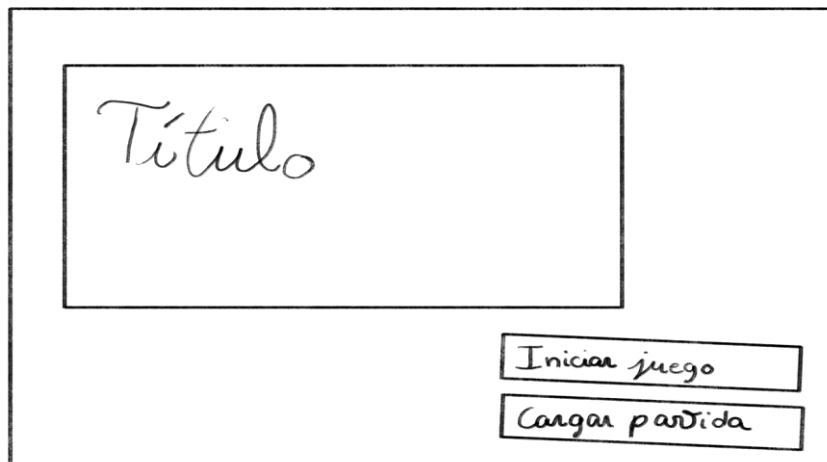
Campo Detalle

Caso de Uso:	CU-04: Exportar Copia de Seguridad (.state)
Actor(es):	Jugador
Descripción:	Aunque el guardado es continuo y automático, el jugador puede descargar un archivo JSON con su progreso actual.
Precondiciones:	Jugador autenticado en la pantalla de ajustes (ajustes.php).
Flujo Principal:	1. El Jugador pulsa "Descargar Partida". 2. El servidor (PHP) consulta todas las tablas de inventario (conejos, recetas, decoraciones) asociadas a su usuarioid. 3. El Sistema formatea los datos en una estructura JSON y fuerza la descarga del fichero BunniesAndTea.state.
Postcondiciones:	El jugador obtiene un archivo físico de respaldo en su equipo.

3.4 MockUP

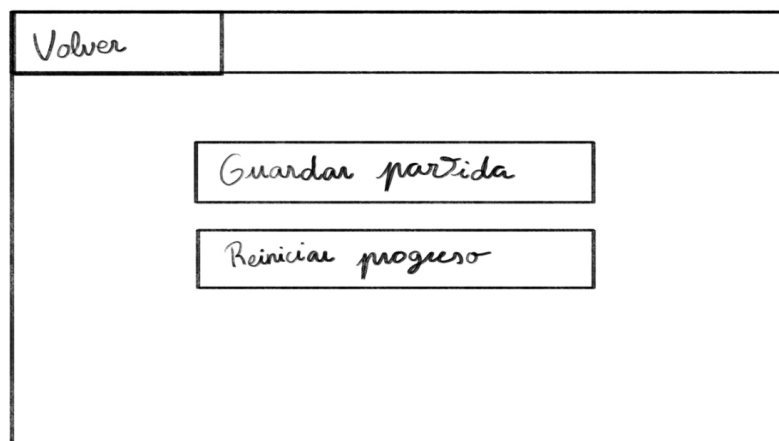
3.4.1 Título

El diseño de la pantalla de entrada es minimalista para centrar la atención del usuario en la acción principal. En lugar de menús complejos, el área central muestra el título del juego y debajo se ubican los controles de acceso. En la versión final, estos botones ("Iniciar Juego" / "Cargar Partida") se sustituirán por un formulario dual de **Registro e Inicio de Sesión** que conectará con la validación de contraseñas de MariaDB, garantizando un acceso seguro a la partida persistente de cada usuario.



3.4.2 Configuración

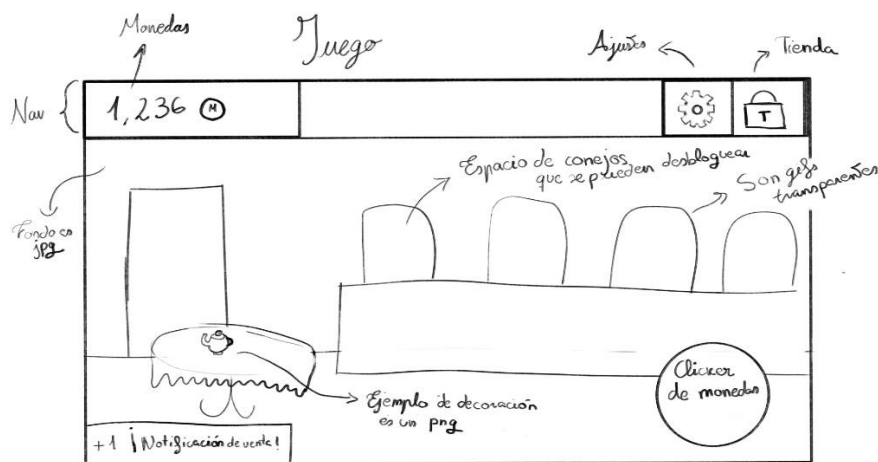
La pantalla de ajustes se ha diseñado como un panel de control secundario de fácil acceso. El botón de retroceso se ubica en la esquina superior izquierda siguiendo los estándares convencionales de navegación web. El cuerpo de la ventana contiene botones grandes y separados para evitar pulsaciones accidentales, especialmente en dispositivos táctiles. Desde aquí, el jugador puede ejecutar acciones críticas como descargar su fichero .state de respaldo o reiniciar su progreso de forma segura.



3.4.3 Juego principal

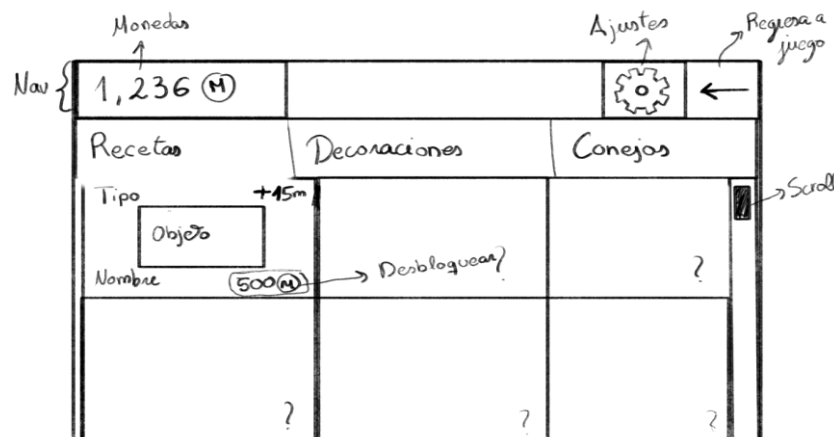
Esta es la pantalla de juego donde el usuario pasará la mayor parte del tiempo, por lo que la interfaz (UI) no debe interferir con la experiencia visual ("Cozy").

- **Barra de navegación (Top-bar):** Se ha diseñado para estar fija en la parte superior. Muestra el saldo de monedas en la izquierda (el dato más consultado) y los accesos a la Tienda y Ajustes a la derecha, agrupando así las acciones administrativas.
- **Área de juego:** Ocupa el centro de la pantalla. Los conejos desbloqueados (GIFs) se irán posicionando dinámicamente en los espacios predefinidos sobre el fondo estático (JPG).
- **Clicker interactivo:** El botón principal para generar monedas se sitúa en la esquina inferior derecha, una zona de fácil acceso para el pulgar en caso de jugar desde un *smartphone*, maximizando así la usabilidad táctil.



3.4.4 Tienda

El diseño de la tienda adopta una estructura de rejilla (*Grid layout*) para mostrar el catálogo completo (Recetas, Decoraciones y Conejos). La barra superior mantiene el contador de monedas fijo para que el jugador pueda ver su saldo real sin tener que volver a la cafetería. Cada tarjeta de producto (card) muestra su nombre, el icono, el beneficio que aporta (ej. +15 monedas/seg) y el botón de compra. Se ha previsto un *scroll* vertical para garantizar que el catálogo completo sea accesible en pantallas pequeñas



4 PLANIFICACIÓN DE ACTUACIONES

Para asegurar el éxito de **Bunnies & Tea**, he seguido una metodología de desarrollo iterativa, dividiendo el trabajo en fases lógicas que permiten tener una versión funcional (MVP) lo antes posible para luego ir añadiendo capas de complejidad y pulido visual.

4.1 SECUENCIACIÓN DE ACTIVIDADES E IDENTIFICACIÓN DE TAREAS

El proyecto se ha estructurado en cuatro fases principales:

1. **Fase 1: Infraestructura y Datos:** Configuración del entorno Docker (Apache/PHP/MariaDB), diseño del esquema de la base de datos y creación de las clases de conexión (PDO) y seguridad (AuthManager).
2. **Fase 2: Lógica de Negocio (Backend):** Implementación de la lógica del juego en PHP: cálculo de producción, validación de compras en la tienda y gestión de sesiones de usuario.
3. **Fase 3: Interfaz e Interactividad (Frontend):** Desarrollo de la estructura HTML, estilos CSS (Flexbox/Grid) e implementación de JavaScript para la comunicación asíncrona (AJAX) y los contadores en tiempo real.
4. **Fase 4: Control de Calidad y Despliegue:** Pruebas de errores, optimización de imágenes y generación de la documentación final (Manual de Usuario e Instalación).

4.2 ESTIMACIONES TEMPORALES

A continuación, se detalla la estimación temporal de las tareas realizadas durante el periodo del proyecto (febrero - mayo 2026):

Fase	Tarea	Semanas
Diseño	Análisis, DAFO, Casos de Uso y Mockups	1-2
Backend	Base de datos, Docker y Clases de Seguridad	3-4
Lógica	Motor de producción y validación de compras	5-6
Frontend	Maquetación CSS y animaciones JS	7-8
Integración	Implementación de AJAX y Sesiones	9-10
QA	Pruebas de carga, depuración y documentación	11-12

4.3 PROCEDIMIENTOS DE EJECUCIÓN

Para garantizar la estabilidad del código y facilitar el mantenimiento futuro, he establecido un marco de trabajo basado en estándares profesionales de desarrollo de software.

Control de versiones y repositorio Utilizaré **Git** para el control de versiones, realizando subidas periódicas al repositorio remoto. Para que el historial de cambios sea legible y profesional, usaré los siguientes prefijos en los mensajes de cada *commit*:

- **[FEAT]**: Para nuevas funcionalidades (ej: "*Añadida la lógica de producción de conejos*").
- **[FIX]**: Para correcciones de errores (ej: "*Arreglado fallo en el contador de monedas*").
- **[DOCS]**: Para cambios en la documentación o comentarios en el código.
- **[STYLE]**: Para cambios estéticos de CSS que no afectan a la lógica.

Protocolo de pruebas y revisión Cada semana realizaré una revisión integral del sistema para detectar posibles regresiones (errores en funciones que ya funcionaban):

- **Pruebas de persistencia**: Validación del ciclo completo de guardado/carga (BBDD y fichero .state).
- **Verificación lógica**: Comprobación de que la producción de monedas (GPS) coincide exactamente con los multiplicadores activos.
- **Test de interfaz**: Revisión de la responsividad en diferentes resoluciones de pantalla.

4.4 RECURSOS Y VALORACIÓN ECONÓMICA

Este apartado detalla los medios necesarios y el coste simulado del proyecto en un entorno laboral real.

4.4.1 Recursos materiales

- Ordenador personal con conexión a internet.
- Entorno Docker con Apache, PHP y MariaDB, ya configurado en clase.
- Programas de edición y diseño gratuitos, como MediBang Paint y diagrams.net.
- Navegadores web (Chrome, Firefox, Opera) para hacer pruebas de visualización.
- Repositorio en GitHub para guardar las versiones del proyecto y copias de seguridad.

4.4.2 Recursos humanos

- El proyecto es individual, por lo tanto, todo el trabajo lo realizo yo: análisis, diseño, desarrollo, pruebas y documentación.
- En caso de dudas técnicas, podré apoyarme en los apuntes de clase o consultar con el profesor, pero el desarrollo completo será propio.

4.4.3 Coste estimado

Concepto	Cantidad	Coste Unitario	Coste Total
Desarrolladora Junior	50 horas	10,00 €/h	500,00 €
Costes Indirectos (Luz, Internet, Amortización)	1 mes	50,00 €	50,00 €
Licencias de Software	-	0,00 €	0,00 €
TOTAL PRESUPUESTADO			550,00 €

4.4.4 Financiación

Al tratarse de un proyecto académico, no necesita financiación externa.

El único coste real es el tiempo invertido y los recursos personales.

4.4.5 Consideraciones para un lanzamiento profesional

Si "Bunnies & Tea" quisiera lanzarse como un producto comercial real, el presupuesto sería muy diferente. Los principales costes para considerar incluirían:

- **Infraestructura (Hosting y BBDD):** Se necesitaría un servidor (VPS o una plataforma en la nube) para alojar la aplicación y una base de datos gestionada que soporte múltiples jugadores, reemplazando el entorno Docker local. Esto implicaría un coste mensual recurrente.
- **Dominio:** Sería necesario adquirir un nombre de dominio propio (ej. bunniesandtea.com), lo cual tiene un coste anual.
- **Activos (Arte y Sonido):** Aunque los gráficos los realizo yo, para un lanzamiento profesional se podría invertir en un diseñador gráfico para pulir los recursos y en un compositor para la música y los efectos de sonido.
- **Marketing y Publicidad:** Se requeriría un presupuesto para dar a conocer el juego en redes sociales, anuncios, etc.
- **Costes Legales:** Gastos para el registro de la marca y la redacción de documentos imprescindibles (Política de Privacidad y Términos de Servicio).

4.5 ASPECTOS LEGALES Y LICENCIAS

Para el desarrollo de **Bunnies & Tea**, se ha garantizado el cumplimiento de la normativa vigente en materia de propiedad intelectual y protección de datos, asegurando que el proyecto sea totalmente legal y ético.

- **Licencias de Software:** Todas las herramientas utilizadas para la creación del proyecto (Visual Studio Code, Docker, MariaDB, Apache) se distribuyen bajo licencias libres y de código abierto como **MIT, GPL o Apache License**. Esto permite su uso en este ámbito académico sin necesidad de adquirir licencias comerciales.
- **Propiedad Intelectual de Recursos Gráficos:** Los activos visuales, incluyendo los personajes (conejos), recetas de productos e interfaz, son de **autoría original**.

y creación propia. Al no utilizar imágenes de terceros, poseo la totalidad de los derechos de explotación y propiedad intelectual del arte del juego.

- **Cumplimiento del RGPD:** Aunque el proyecto es una simulación, el sistema de gestión de usuarios se ha diseñado siguiendo los principios del **Reglamento General de Protección de Datos (RGPD)**. Las contraseñas se almacenan mediante algoritmos de cifrado (*hashing*) y no se recopila información personal sensible más allá del email para el inicio de sesión.

4.6 GESTIÓN DE RIESGOS

En todo desarrollo de software existen imprevistos. Para este proyecto, he identificado los riesgos más críticos y he diseñado un plan de contingencia para cada uno:

Riesgo Detectado	Tipo	Nivel de Impacto	Plan de Contingencia (Solución)
Pérdida de la base de datos	Técnico	Alto	Uso de <i>Docker Volumes</i> para persistencia y exportación diaria de un script backup.sql para restaurar los datos en minutos.
Corrupción del fichero .state	Técnico	Alto	Implementar una validación estricta del JSON en PHP antes de cargar nada. Si el archivo está mal, mostraré un error en vez de romper el juego.
Retraso en el diseño de los conejos	Gestión	Medio	Si no me da tiempo a dibujar todos los conejos animados, usaré siluetas o imágenes estáticas (<i>placeholders</i>) para no detener la programación.
Desincronización JS/PHP	Técnico	Medio	Si el contador de monedas visual no coincide con el de la base de datos, forzaré una recarga de los datos del servidor cada 60 segundos.
Falta de tiempo en la entrega	Gestión	Alto	Priorizaré las funciones básicas (MVP): primero que funcione la tienda y el guardado. Las animaciones decorativas se dejarán para el final si sobra tiempo.

5 SEGUIMIENTO Y EVALUACIÓN

5.1 EVALUACIÓN

5.1.1 Evaluación inicial

Antes de iniciar la codificación, se ha realizado una auditoría técnica para asegurar la viabilidad del proyecto, validando los siguientes puntos:

- **Infraestructura y Entorno:** Verificación del despliegue de contenedores Docker (Apache, PHP y MariaDB) y su correcta interconexión.
- **Integridad del Diseño:** Revisión del Diagrama Entidad-Relación y los Mockups para confirmar que cubren todos los requisitos funcionales.
- **Disponibilidad de Recursos:** Configuración del entorno de desarrollo (VS Code) y sincronización del repositorio Git.

5.1.2 Planificación de las revisiones

Se han establecido tres hitos de control para monitorizar el progreso y detectar desviaciones de forma temprana:

- **Hito 1: Revisión de Backend:** Validación de la persistencia en base de datos y generación del fichero .state. Es un punto de control crítico antes de iniciar el diseño visual.
- **Hito 2: Revisión Visual:** Evaluación de la maquetación responsiva y la velocidad de carga de los recursos gráficos en diferentes dispositivos.
- **Hito 3: Revisión Final:** Simulacro de partida real ("User Acceptance Test") para garantizar que el flujo de juego es fluido y libre de errores.

5.2 INDICADORES DE CALIDAD

Para garantizar que la lógica de negocio sea robusta y libre de errores, he integrado la biblioteca PHPUnit en el flujo de desarrollo del backend. Mi objetivo principal con estas pruebas automatizadas es asegurar que los cálculos críticos del juego, como la producción pasiva de monedas o los descuentos por suciedad, funcionen exactamente como se espera. En lugar de probar cada compra manualmente en el navegador, he programado tests que verifican de forma aislada que las funciones del controlador devuelven los valores correctos. Esto me permite detectar fallos al instante cada vez que introduzco un nuevo tipo de conejo o utensilio en la base de datos. Al ejecutar estos tests directamente desde el contenedor de Docker, me aseguro de que el entorno de pruebas sea idéntico al de producción, lo que aporta una capa de seguridad extra y profesionalidad a la fase de implementación final.

Se ha definido la siguiente batería de pruebas funcionales para asegurar el correcto comportamiento de las mecánicas principales:

ID	Funcionalidad	Resultado Esperado	Criterio "PASS" (Aprobado)	Criterio "FAIL" (Suspenso)
CP-01	Carga de partida	Restauración exacta de monedas y activos desde el archivo JSON.	Sincronización del 100% entre el JSON y la pantalla.	Los contadores aparecen a 0 o la carga se congela.
CP-02	Mecánica Clicker	Incremento del saldo tras la interacción del usuario.	Actualización instantánea (JS) y sincronización BBDD (AJAX).	Retraso visible (>200ms) en la actualización del número.
CP-03	Compra en Tienda	Validación del saldo y actualización del inventario.	El servidor bloquea compras si no hay saldo real.	Permite comprar sin saldo o el objeto no aparece.
CP-04	Producción Auto	Generación de ingresos pasivos por segundo (GPS).	El contador sube automáticamente según la lógica del servidor.	La producción se detiene al minimizar el navegador.

5.2.1 Batería de procedimientos de pruebas

Complementariamente, se realizan pruebas técnicas semanales:

- **Prueba de estrés:** Clics masivos para verificar la estabilidad de las peticiones al servidor.
- **Prueba de "navegador limpio":** Ejecución en modo incógnito para evitar falsos positivos por la caché.
- **Validación de archivos:** Intento de carga de ficheros corruptos para probar la tolerancia a fallos del sistema.

5.3 GESTIÓN DE INCIDENCIAS

Se utiliza un registro sistemático de errores para controlar el estado de los fallos detectados:

ID	Fecha	Descripción del error	Prioridad	Estado	Solución aplicada
INC-00	DD/MM	(Ejemplo: El botón de comprar no resta monedas)	Alta	Pendiente	-

5.3.1 Sistematización y temporalización

Las incidencias se clasifican según su impacto para determinar su prioridad de resolución:

- **Incidencia Crítica (Alta):** Errores que impiden jugar o provocan pérdida de datos. Resolución inmediata antes de continuar el desarrollo.
- **Incidencia Funcional (Media):** Errores en mecánicas específicas que no bloquean el sistema. Resolución programada para el siguiente hito.
- **Incidencia Estética (Baja):** Defectos visuales o de interfaz. Resolución en la fase final de pulido.

5.4 GESTIÓN DE CAMBIOS

5.4.1 Formato de registro de cambios

Cualquier desviación sobre el diseño original se documenta para justificar la evolución del proyecto:

ID	Fecha	Descripción del cambio	Motivo	Decisión
CAM-00	DD/MM	(Ej: Eliminar la música de fondo)	Falta de tiempo	Aprobado

5.4.2 Análisis de impacto

Antes de aprobar un cambio, se evalúa su impacto en el cronograma (**Impacto Temporal**), la estabilidad del código (**Riesgo Técnico**) y su necesidad para el **MVP** (Producto Mínimo Viable).

5.5 DESARROLLO TÉCNICO E IMPLEMENTACIÓN

5.5.1 Backend y Lógica de Negocio

Para el desarrollo del servidor he utilizado PHP 8.2, centrándome en crear una estructura que sea tanto funcional como segura. La comunicación con la base de datos MariaDB se realiza exclusivamente a través del driver PDO. Al usar consultas preparadas, me aseguro de que la aplicación esté protegida contra inyecciones SQL, garantizando que los datos de los usuarios y su progreso en la cafetería no puedan ser comprometidos.

Uno de los pilares de la implementación ha sido la validación de la economía en el backend. En un juego de este tipo, es fácil caer en el error de confiar en el saldo que muestra el navegador, pero en mi proyecto el servidor siempre tiene la última palabra. Cada vez que el usuario intenta realizar una compra, el controlador de PHP verifica el saldo real directamente en la base de datos. De esta forma, si un usuario intentara trucar sus monedas mediante la consola de JavaScript para comprar un objeto sin tener dinero suficiente, el backend detectaría la discrepancia y bloquearía la operación.

Finalmente, para gestionar el acceso de forma profesional, he implementado un sistema de sesiones nativas y protección de cuentas. Las contraseñas nunca se guardan en texto plano, sino que se cifran utilizando `password_hash`. Esto me permite asegurar que cada jugador tenga su propia experiencia privada y que su progreso se recupere de forma íntegra y segura en cada inicio de sesión.

5.5.2 Frontend e Interfaz de Usuario

Para el desarrollo del cliente, mi objetivo ha sido crear una interfaz reactiva que combine la ligereza del código nativo con funciones visuales potentes. He desarrollado el núcleo del juego utilizando JavaScript Vanilla, lo que me permite un control total sobre el rendimiento y evita dependencias innecesarias. Sin embargo, para la parte analítica y el seguimiento del progreso, he integrado la librería Chart.js. Con esto he conseguido que el jugador pueda visualizar gráficamente su evolución y su producción de monedas a lo largo del tiempo, dándole un toque más profesional y técnico a la interfaz.

La interactividad se apoya totalmente en la Fetch API para gestionar las peticiones asíncronas (AJAX). Gracias a esto, el juego funciona de manera fluida y sin recargas de página: cada vez que el usuario compra una mejora o interactúa con la cafetería, los datos se envían al servidor en segundo plano y la respuesta se procesa mediante la manipulación del DOM. Esto permite que tanto los contadores de monedas como los gráficos de Chart.js se actualicen en tiempo real, manteniendo la inmersión del jugador en todo momento.

En cuanto al diseño visual, he trabajado con CSS3 para asegurar que la estética "Cozy" se mantenga en cada rincón de la aplicación. He utilizado sistemas de Grid y Flexbox para organizar la tienda y los elementos del juego de forma equilibrada. Además, me he asegurado de que los estilos de los gráficos analíticos se adapten a la paleta de colores pastel y a la tipografía del proyecto, logrando que el uso de librerías externas no rompa la armonía visual relajante que buscaba desde el principio.

5.5.3 Ampliación de contenido y sistema de logros

Para darle más profundidad al juego, he decidido añadir un sistema de logros y ampliar el contenido disponible en la tienda. Los logros están integrados en el menú de ajustes y funcionan mediante disparadores en el servidor que comprueban hitos específicos, como alcanzar un total de monedas o contratar a un número determinado de conejos. Esto me ha permitido programar una lógica de control más avanzada en el backend para gestionar el progreso del usuario. Además, la expansión de la tienda con nuevos personajes y herramientas me ha servido para validar que el diseño de la base de datos es realmente escalable; he podido introducir estos nuevos elementos sin necesidad de modificar la estructura SQL original, lo que facilita mucho el crecimiento del proyecto sin generar errores de compatibilidad.

5.6 MANUAL DE USO Y DESPLIEGUE DEL SISTEMA

5.6.1 Manual para usuario

Para que el proyecto sea totalmente portable y fácil de probar en cualquier entorno de desarrollo, he optado por una infraestructura basada en contenedores. Utilizo Docker y Docker Compose para orquestar tanto el servidor web Apache con PHP 8.2 como el servicio de base de datos MariaDB. He configurado el despliegue para que sea prácticamente automático: una vez clonado el repositorio de GitHub, solo hace falta ejecutar el comando `docker-compose up -d` desde la terminal en la carpeta raíz. Este proceso se encarga de levantar los servicios y configurar los volúmenes necesarios para que los datos de los usuarios no se pierdan al apagar los contenedores. Una vez que el sistema está activo, el juego es accesible directamente desde el navegador a través de `http://localhost`.

En cuanto a la experiencia del usuario, el funcionamiento de Bunnies & Tea es sencillo e intuitivo. Lo primero que debe hacer el jugador es registrarse para crear su propia cafetería persistente en la base de datos. Una vez iniciada la sesión, la mecánica principal consiste en generar monedas, ya sea de forma manual haciendo clic o invirtiendo en la Tienda para automatizar el proceso. En la tienda he programado diferentes tipos de conejos que generan ingresos de forma pasiva, lo que permite que el saldo siga subiendo incluso cuando no se está clicando.

Un aspecto importante que he integrado en el manual de juego es el sistema de mantenimiento. Para evitar que el jugador se limite a dejar la ventana abierta sin interactuar, he implementado una mecánica de suciedad: si la cafetería no se limpia regularmente, aparece un aviso y la producción de monedas cae un 50% automáticamente. Por último, para mayor seguridad del progreso, el usuario tiene la opción de exportar su partida en un archivo con extensión `.state`, que sirve como copia de seguridad física para poder retomar su progreso en cualquier otro momento o navegador.

5.6.2 Manual de administrador

Para la gestión técnica, el administrador puede monitorizar el sistema mediante docker logs para detectar errores en la Fetch API. Además, es posible realizar ajustes de balanceo en la economía accediendo directamente al contenedor de MariaDB para actualizar los precios en la tabla conejo o utensilio sin necesidad de detener el servicio.

6 EVALUACIÓN Y CONCLUSIONES FINALES

La realización de este proyecto ha supuesto un reto integral que ha ido más allá de la simple programación. A nivel de resultados, la arquitectura MVC ha demostrado ser la base idónea para gestionar un idle game, permitiendo una separación limpia entre la lógica económica del servidor y la interactividad del cliente.

Es importante señalar que, aunque el motor principal es totalmente operativo, el proyecto se encuentra actualmente en una fase de revisión técnica y corrección de la documentación. Durante las próximas semanas, el trabajo no solo se centrará en picar código, sino en blindar el sistema mediante PHPUnit para asegurar que los cálculos de monedas sean exactos y en organizar toda la memoria técnica para que refleje fielmente la implementación real.

De cara a la entrega definitiva, soy consciente de que aún quedan aspectos por pulir, como la finalización de algunos activos visuales, la integración total de las gráficas de Chart.js y un balanceo más fino del sistema de logros. No obstante, la base actual es sólida y cumple con todos los requisitos de viabilidad planteados, dejando el camino despejado para finalizar los detalles estéticos y analíticos que faltan.